Real-Time Operating System(RTOS)

Today Agenda

- Introduction to Real-Time System
- FreeRTOS
- Task Management
- Task Scheduling and Example
- Reference Reading: *Intro to FreeRTOS.pdf(1-103), Introduction to Real-Time.pdf*

Introduction to Real-Time

- Real-time System:
  - is one in which the correctness of the computations not only **depends on their logical correctness**, but **also on the time** at which the result is produced. In other words, a late answer is a wrong answer.
- Example:
  - a computer-controlled machine on the production line at a bottling plant.
  - The machine's function is simply to cap each bottle as it passes within the machine's field of motion on a continuously moving conveyor belt.
  - If the machine operates too quickly, the bottle won't be there yet.
  - If the machine operates too slowly, the bottle will be too far along for the machine to reach it.

Introduction to Real-Time

- Real-time System:
  - is one in which the correctness of the computations not only **depends on their logical correctness**, but **also on the time** at which the result is produced. In other words, a late answer is a wrong answer.
- Example:
  - a computer-controlled machine on the production line at a bottling plant.
  - The machine's function is simply to cap each bottle as it passes within the machine's field of motion on a continuously moving conveyor belt.
  - If the machine operates too quickly, the bottle won't be there yet.
  - If the machine operates too slowly, the bottle will be too far along for the machine to reach it.

Introduction to Real-Time

- Deadline Driven:
  - This window of opportunity imposes timing constraints on the operation of the machine. Applications with these kinds of timing constraints are considered real time. In this case, the timing constraints are in the form of a period and deadline.
- Example:
  - Consider a cruise control mechanism on an automobile. The basic operation of cruise control is to keep the speed of the vehicle constant.
  - Suppose the driver selects 60mph as the desired speed.
  - If the vehicle is going slower than 60mph, then the embedded computer sends a signal to the engine controller to accelerate.
  - If the vehicle is going faster than 60mph, it sends a signal to decelerate.
  - ? A question to ask is: how often does the computer check if the current speed is too slow or too fast?

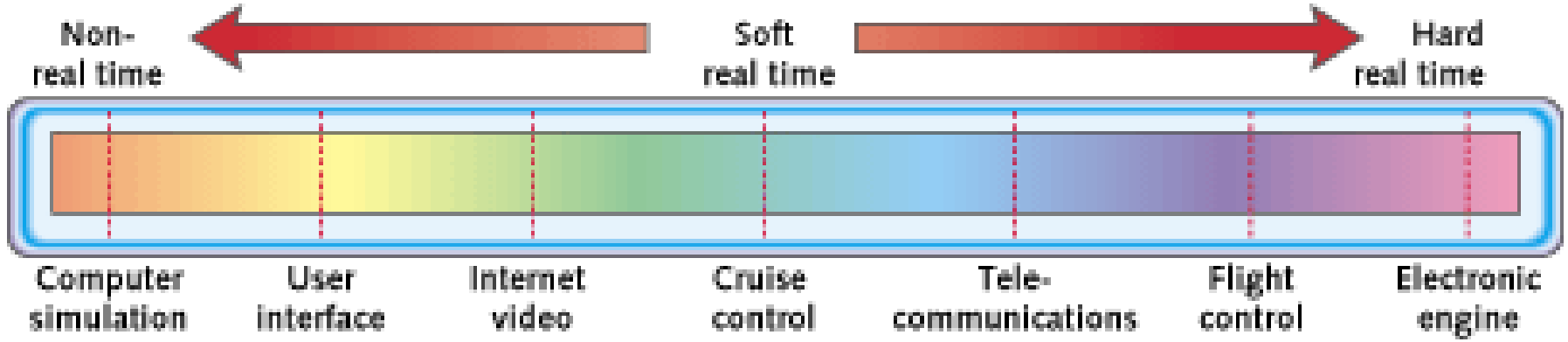
Introduction to Real-Time

- Aperiodic Tasks:
  - Aperiodic tasks, also called aperiodic servers, respond to randomly arriving events.
- Example:
  - Consider anti-lock braking.
  - If the driver presses the brake pedal, the car must respond very quickly.
  - The response time is the time between the moment the brake pedal is pressed, and the moment the anti-lock braking software actuates the brakes.

Introduction to Real-Time

- Hard or Soft:
  - At one end of the spectrum is non-real-time, where there are no important deadlines (meaning all deadlines can be missed). The other end is hard real-time, where no deadlines can be missed.

Figure 1: The real-time spectrum



Introduction to Real-Time

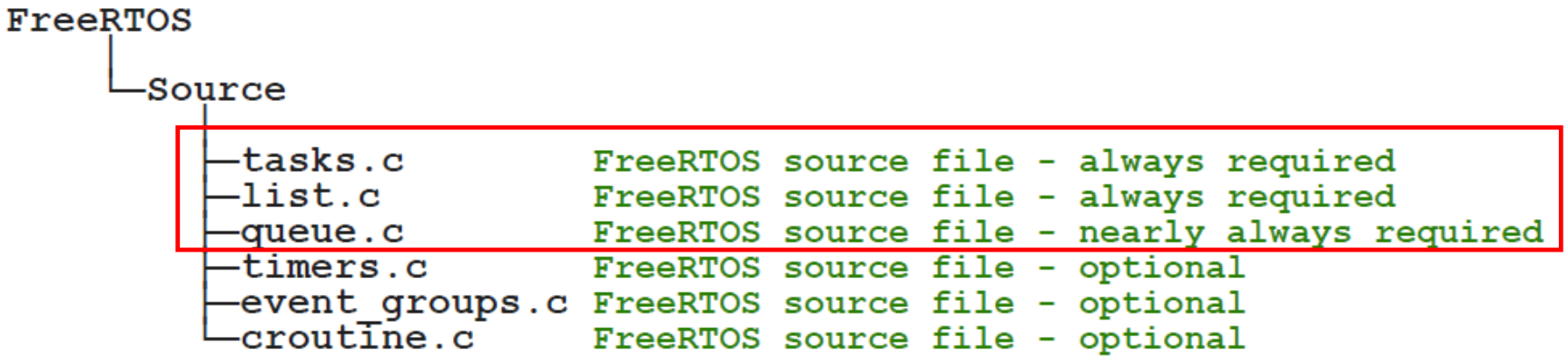
- Predictability:
  - a system whose timing behavior is always within an acceptable range.
  - the period, deadline, and worst-case execution time of each task need to be known to create a predictable system.
- Deterministic:
  - Is a special case of a predictable system.
  - Not only is the timing behavior within a certain range, but that timing behavior can be pre-determined.

FreeRTOS

- Embedded operating system allow multiple tasks to execute with a scheduler to switch between tasks.
- Features included:
  - Priority-based multitasking capability
  - Queues to communicate between multiple tasks
  - Semaphores to manage resource sharing between multiple tasks
  - Utilities to view CPU, resource and stack utilization

FreeRTOS Source Files

- The core FreeRTOS source code is contained in just two C files: tasks.c and list.c located in the FreeRTOS/Source directory.
- queue.c provides both queue and semaphore services and is nearly always required.



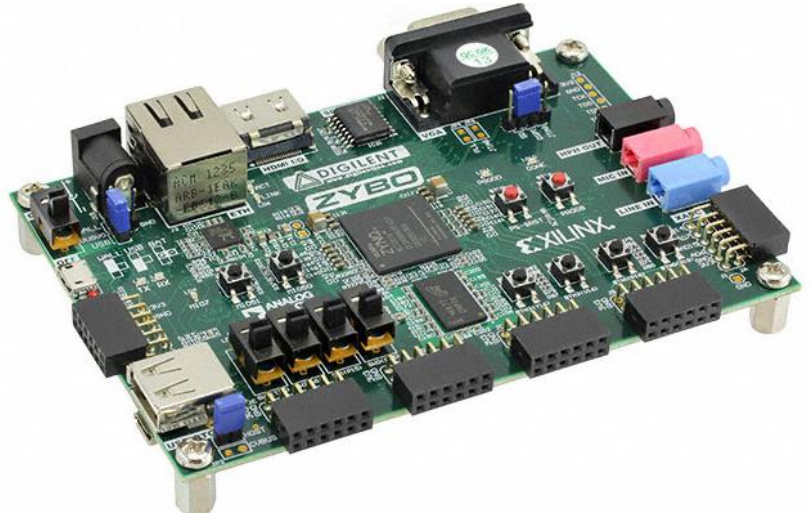
FreeRTOS Source Files

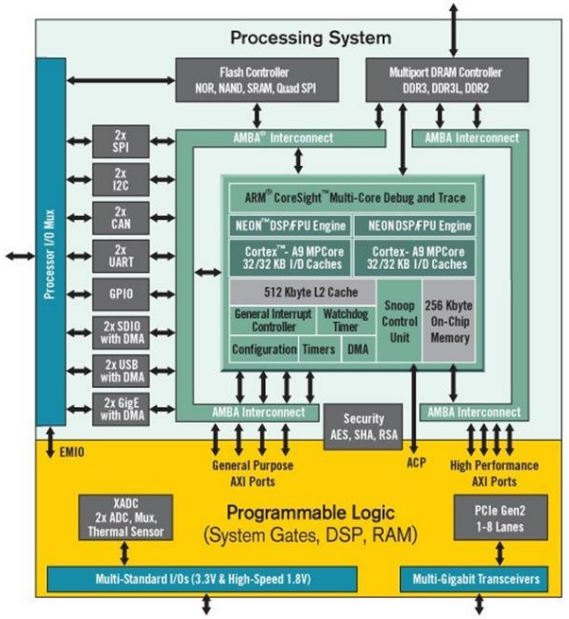
- timers.c provides software timer functionality and needs to be included if software timers are used.
- event_groups.c and croutine.c are advanced features and not used here.

```
FreeRTOS
    └─Source
        ├─tasks.c          FreeRTOS source file - always required
        ├─list.c           FreeRTOS source file - always required
        ├─queue.c          FreeRTOS source file - nearly always required
        ├─timers.c         FreeRTOS source file - optional
        ├─event_groups.c   FreeRTOS source file - optional
        └─croutine.c       FreeRTOS source file - optional
```

FreeRTOS Implementation inXilinx Vivado and SDK

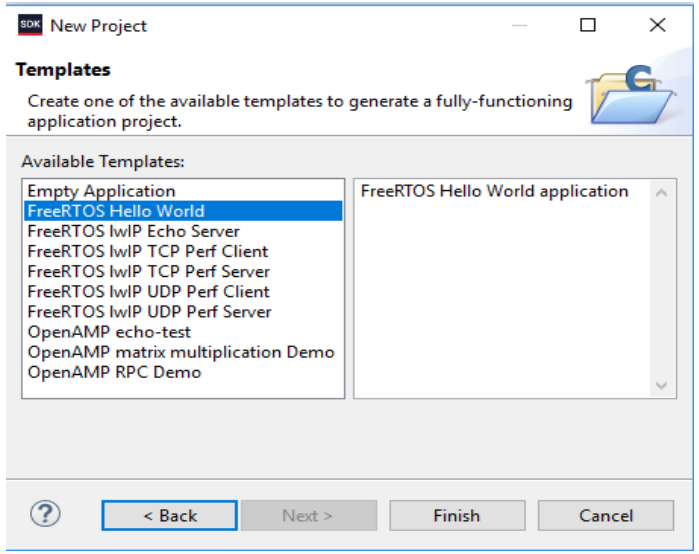
- Xilinx Vivado has incorporated FreeRTOS into SDK which obviates what was once a difficult process of software integration.
- Prior to now the FreeRTOS operating system files were downloaded separately and files had to be copied-and-pasted into SDK before the application could be configured.
- The FreeRTOS implementation is now fully integrated with the Digilent Zybo Board Support Package (BSP) in the SDK.

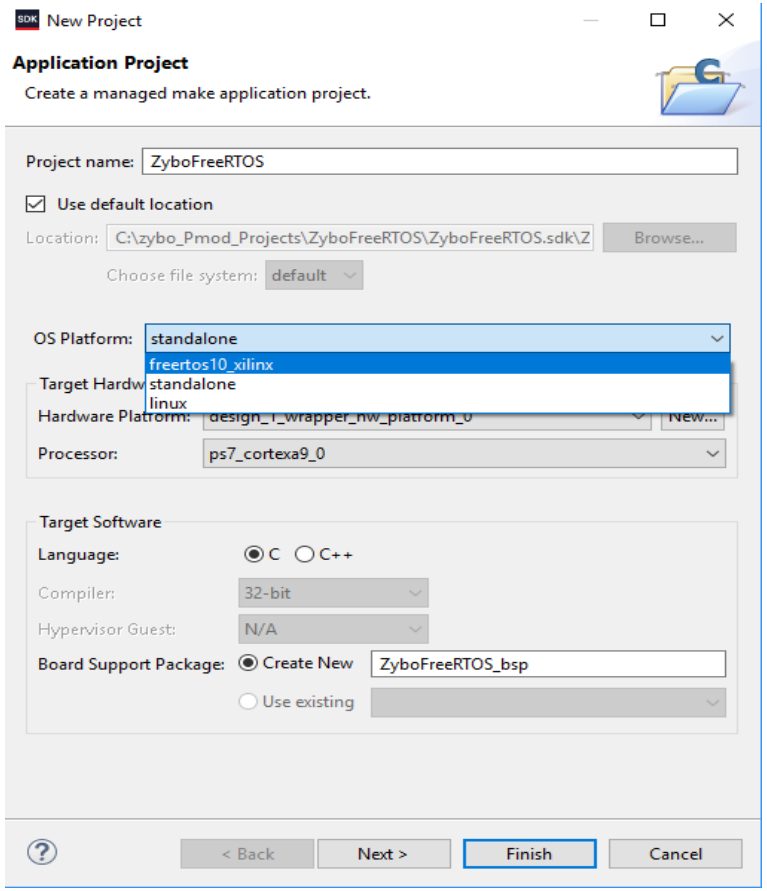




FreeRTOS Implementation inXilinx Vivado and SDK

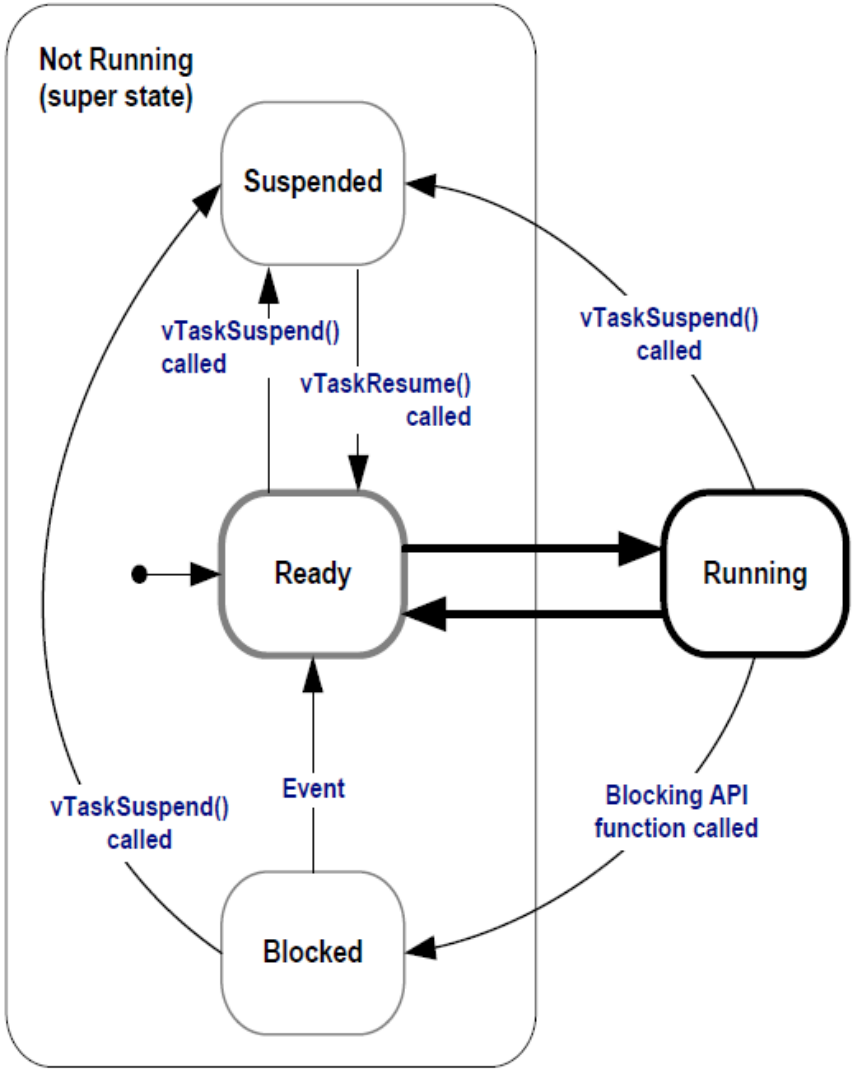
- FreeRTOS version 10 is now available as the OS Platform in SDK. The other OSs are standalone (or bare metal) and Linux.





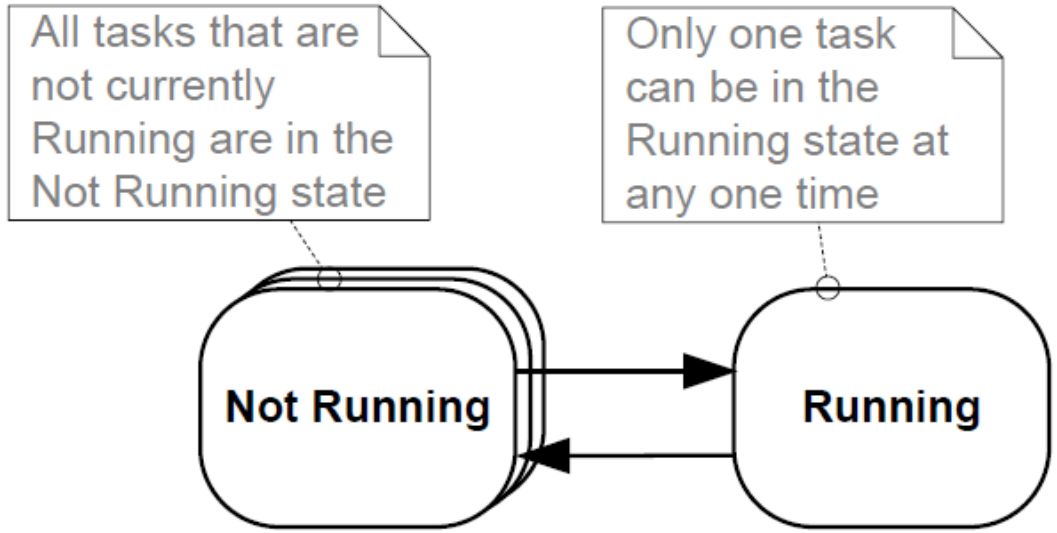
Task Status

- Ready:
  - are able to run but are not currently in the Running state
- Running
- Blocked:
  - A task that is waiting for an even
  - Temporal (time-related) events
  - Synchronization events
- Suspend:
  - are not available to the FreeRTOS scheduler.



Task Status

- An application can consist of many tasks. If the processor running the application contains a single core, then only one task can be executing at any given time.
- This implies that a task can exist in either a Running state executing the task or a simplistic Not Running state.



FreeRTOS Data Types

- Two port specific data types: TickType_t and BaseType_t
  - TickType_t is used to hold the tick count value and to specify times and is unsigned 32-bit.
  - BaseType_t is used for return types that take only a very limited range of values and for pdTrue and pdFALSE.
- Tick count: the number of tick interrupts
- Tick period: the time between two tick interrupts
- pdTRUE, pdFALSE, pdPASS, pdFAIL

Create Task

- Define a task:
  - return void and take a void pointer parameter.
  - Each task is a small program in its own right and has an entry point which will normally run forever within an infinite loop will not exit.

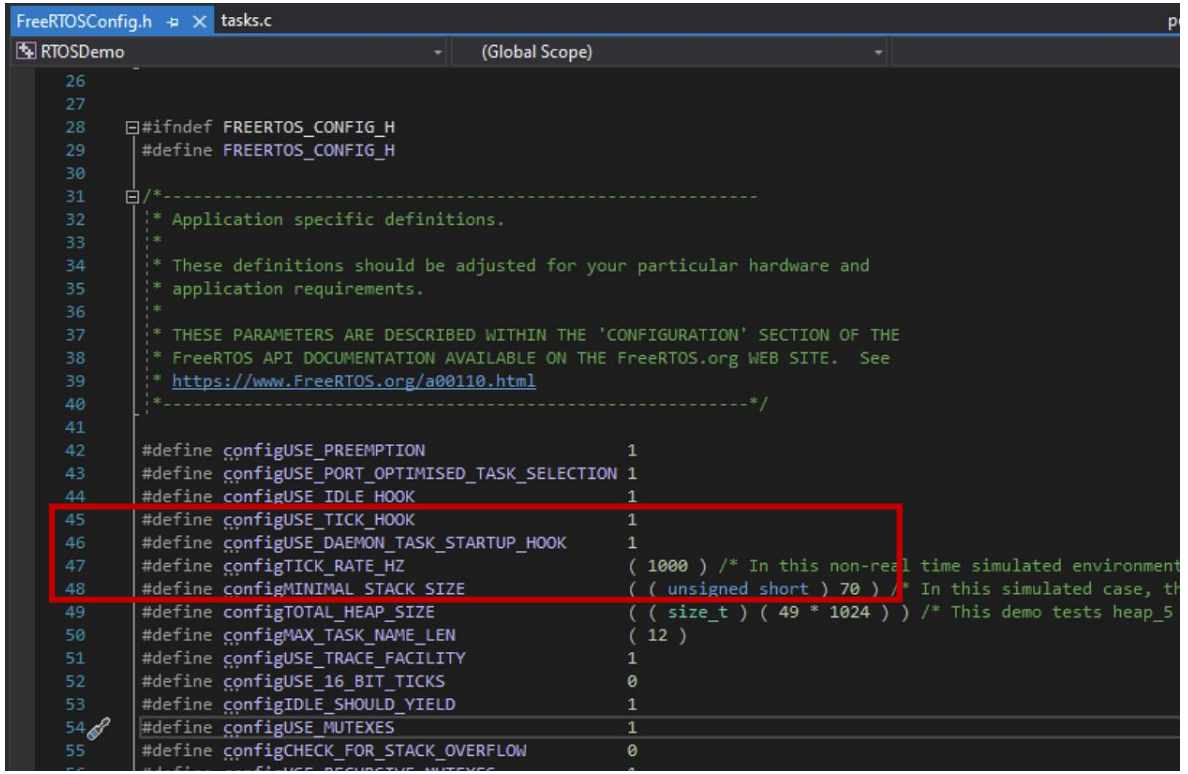
```
void ATaskFunction( void *pvParameters )
{
/* Variables can be declared just as per a normal function.  Each instance of a task
created using this example function will have its own copy of the lVariableExample
variable.  This would not be true if the variable was declared static – in which case
only one copy of the variable would exist, and this copy would be shared by each
created instance of the task. (The prefixes added to variable names are described in
section 1.5, Data Types and Coding Style Guide.) */
int32_t lVariableExample = 0;

    /* A task will normally be implemented as an infinite loop. */
    for( ;; )
    {
        /* The code to implement the task functionality will go here. */
    }

    /* Should the task implementation ever break out of the above loop, then the task
    must be deleted before reaching the end of its implementing function.  The NULL
    parameter passed to the vTaskDelete() API function indicates that the task to be
    deleted is the calling (this) task.  The convention used to name API functions is
    described in section 1.5, Data Types and Coding Style Guide. */
    vTaskDelete( NULL );
}
```

Create Task

- Create a task:
  - pvParameters: Task functions accept a parameter of type pointer to void (void*). The value assigned to pvParameters is the value passed into the task.
  - uxPriority: Defines the priority at which the task will execute. Priorities can be assigned from 0, the lowest priority, to (configMAX_PRIORITIES – 1), which is the highest priority.
  - pxCreatedTask: used to pass a handle to the task being Created which is used to reference the task in API calls that, for example, change the task priority or delete the task. If not used then pxCreatedTask can be set to NULL.

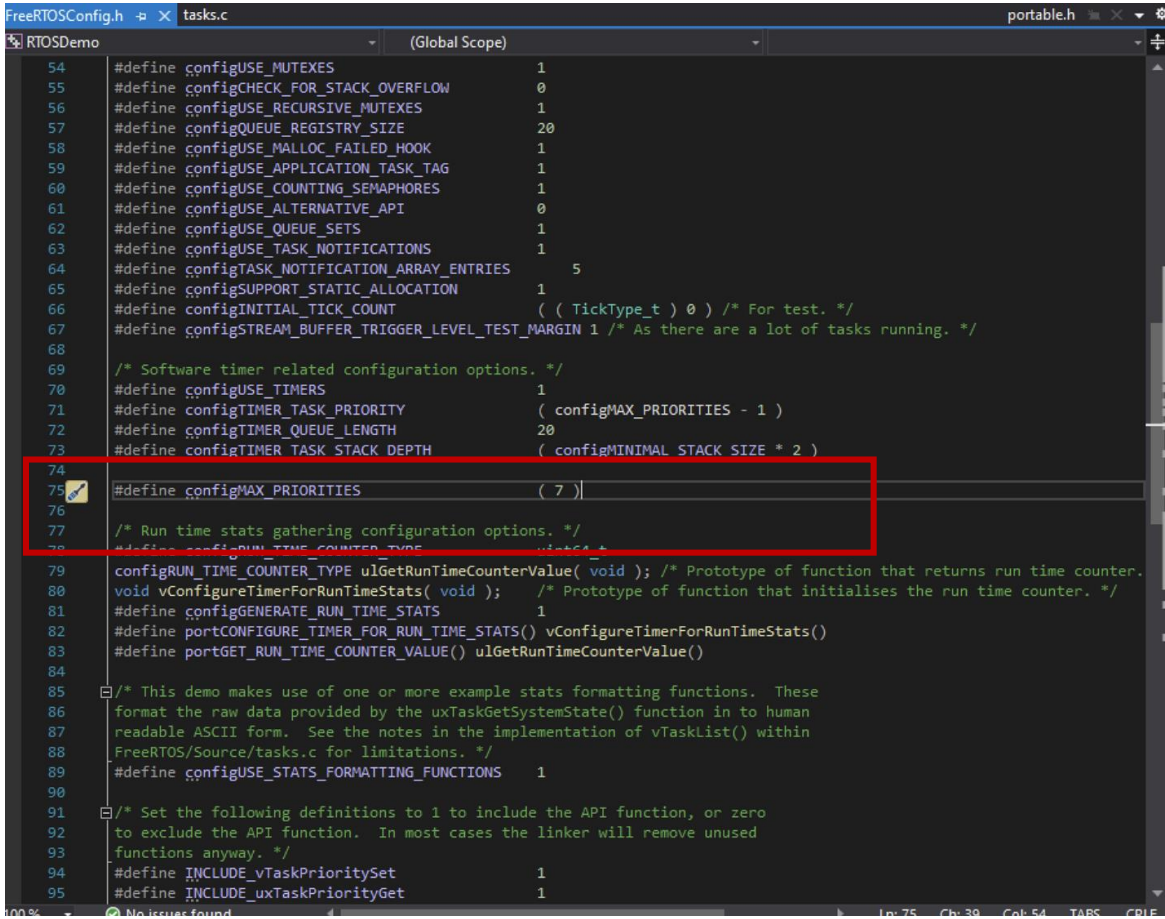
Create Task

- Tick:
  - Tick rate: #define configTICK_RATE_HZ 100
  - Convert time in millisecond to tick: pdMS_TO_TICKS()
  - pdMS_TO_TICKS() cannot be used if the tick frequency is above 1 kHz (that is if configTICK_RATE_HZ is greater than 1000). Why?



Create Task

- Priority:
  - The maximum number of priorities available is set by configMAX_PRIORITIES constant in FreeRTOSConfig.h.
  - #define configMAX_PRIORITIES: integer
  - The range of available priorities is 0 to (configMAX_PRIORITIES – 1).
  - <=32



Create Task--Example

Includes, defines and instantiations:

```
/* FreeRTOS.org
#include "FreeRT
#include "task.h"

/* Used as a loop
#define mainDEL

/* The task functic
void vTask1( void
void vTask2( void
```

```
int main( void )
{
    /* Create one of the two tasks.  Note that a real application should check
    the return value of the xTaskCreate() call to ensure the task was created
    successfully. */
    xTaskCreate(    vTask1,  /* Pointer to the function that implements the task. */
                    "Task 1",/* Text name for the task.  This is to facilitate
                             debugging only. */
                    1000,    /* Stack depth - small microcontrollers will use much
                             less stack than this. */
                    NULL,    /* This example does not use the task parameter. */
                    1,       /* This task will run at priority 1. */
                    NULL );  /* This example does not use the task handle. */

    /* Create the other task in exactly the same way and at the same priority. */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    /* Start the scheduler so the tasks start executing. */
    vTaskStartScheduler();

    /* If all is well then main() will never reach here as the scheduler will
    now be running the tasks.  If main() does reach here then it is likely that
    there was insufficient heap memory available for the idle task to be created.
    Chapter 2 provides more information on heap memory management. */
    for( ;; );
}
```

Create Task--Example

Main function:

```
int main( void )
{
    /* Create one of the two tasks.  Note that a real application should check
    the return value of the xTaskCreate() call to ensure the task was created
    successfully. */
    xTaskCreate(    vTask1,  /* Pointer to the function that implements the task. */
                    "Task 1",/* Text name for the task.  This is to facilitate
                             debugging only. */
                    1000,    /* Stack depth - small microcontrollers will use much
                             less stack than this. */
                    NULL,    /* This example does not use the task parameter. */
                    1,       /* This task will run at priority 1. */
                    NULL );  /* This example does not use the task handle. */

    /* Create the other task in exactly the same way and at the same priority. */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    /* Start the scheduler so the tasks start executing. */
    vTaskStartScheduler();

    /* If all is well then main() will never reach here as the scheduler will
    now be running the tasks.  If main() does reach here then it is likely that
    there was insufficient heap memory available for the idle task to be created.
    Chapter 2 provides more information on heap memory management. */
    for( ;; );
}
```

Create Task--Example

Task1 :

```
void vTask1( void *pvParameters )
{
const char *pcTaskName = "Task 1 is running\r\n";
volatile uint32_t ul; /* volatile to ensure ul is not optimized away. */

    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; )
    {
        /* Print out the name of this task. */
        vPrintString( pcTaskName );

        /* Delay for a period. */
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ )
        {
            /* This loop is just a very crude delay implementation.  There is
            nothing to do in here.  Later examples will replace this crude
            loop with a proper delay/sleep function. */
        }
    }
}
```

Create Task--Example

Task 2

```
int main( void )
{
    /* Create one of the two tasks.  Note that a real application should check
    the return value of the xTaskCreate() call to ensure the task was created
    successfully. */
    xTaskCreate(    vTask1,  /* Pointer to the function that implements the task. */
                    "Task 1",/* Text name for the task.  This is to facilitate
                             debugging only. */
                    1000,    /* Stack depth - small microcontrollers will use much
                             less stack than this. */
                    NULL,    /* This example does not use the task parameter. */
                    1,       /* This task will run at priority 1. */
                    NULL );  /* This example does not use the task handle. */

    /* Create the other task in exactly the same way and at the same priority. */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    /* Start the scheduler so the tasks start executing. */
    vTaskStartScheduler();

    /* If all is well then main() will never reach here as the scheduler will
    now be running the tasks.  If main() does reach here then it is likely that
    there was insufficient heap memory available for the idle task to be created.
    Chapter 2 provides more information on heap memory management. */
    for( ;; );
}
```

FreeRTOS Example of Task Creation

This example demonstrates the steps needed to create two simple tasks and then start the tasks executing with *main()*.

The tasks print out a string periodically, using a crude null loop to create the delay. Both tasks are created at the same priority and are identical except for the string they print out.

The main() function creates the tasks before starting the scheduler.

FreeRTOS Example of Task Creation

Includes, defines and instantiations:

```
/* FreeRTOS.org includes. */
#include "FreeRTOS.h"
#include "task.h"

/* Used as a loop counter to create a very crude delay. */
#define mainDELAY_LOOP_COUNT                ( 0xffffff )

/* The task functions. */
void vTask1( void *pvParameters );
void vTask2( void *pvParameters );
```

Example 001

FreeRTOS Example of Task Creation

main()

```
int main( void )
{
    /* Create one of the two tasks.  Note that a real application should check
    the return value of the xTaskCreate() call to ensure the task was created
    successfully. */
    xTaskCreate(    vTask1,  /* Pointer to the function that implements the task. */
                    "Task 1",/* Text name for the task.  This is to facilitate
                             debugging only. */
                    1000,    /* Stack depth - small microcontrollers will use much
                             less stack than this. */
                    NULL,    /* This example does not use the task parameter. */
                    1,       /* This task will run at priority 1. */
                    NULL );  /* This example does not use the task handle. */

    /* Create the other task in exactly the same way and at the same priority. */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    /* Start the scheduler so the tasks start executing. */
    vTaskStartScheduler();

    /* If all is well then main() will never reach here as the scheduler will
    now be running the tasks.  If main() does reach here then it is likely that
    there was insufficient heap memory available for the idle task to be created.
    Chapter 2 provides more information on heap memory management. */
    for( ;; );
}
```

Listing 16

FreeRTOS Example of Task Creation

Task 1:

```
void vTask1( void *pvParameters )
{
const char *pcTaskName = "Task 1 is running\r\n";
volatile uint32_t ul; /* volatile to ensure ul is not optimized away. */

    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; )
    {
        /* Print out the name of this task. */
        vPrintString( pcTaskName );

        /* Delay for a period. */
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ )
        {
            /* This loop is just a very crude delay implementation.  There is
            nothing to do in here.  Later examples will replace this crude
            loop with a proper delay/sleep function. */
        }
    }
}
```

Listing 14

FreeRTOS Example of Task Creation

Task 2:

```
void vTask2( void *pvParameters )
{
const char *pcTaskName = "Task 2 is running\r\n";
volatile uint32_t ul; /* volatile to ensure ul is not optimized away. */

    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; )
    {
        /* Print out the name of this task. */
        vPrintString( pcTaskName );

        /* Delay for a period. */
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ )
        {
            /* This loop is just a very crude delay implementation.  There is
            nothing to do in here.  Later examples will replace this crude
            loop with a proper delay/sleep function. */
        }
    }
}
```

Task Management

- Suspend and resume task:
  - void vTaskSuspend(TaskHandle_t pxTaskToSuspend);
  - void vTaskResume(TaskHandle_t pxTaskToResume);

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        uint16_t usStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t *pxCreatedTask );
```

Task Management

- Task Delay
  - vTaskDelayUntil()

```
void vTaskDelay( TickType_t xTicksToDelay );
```

- Delay Until:
  - vTaskDelayUntil()

```
void vTaskDelayUntil( TickType_t * pxPreviousWakeTime, TickType_t xTimeIncrement );
```

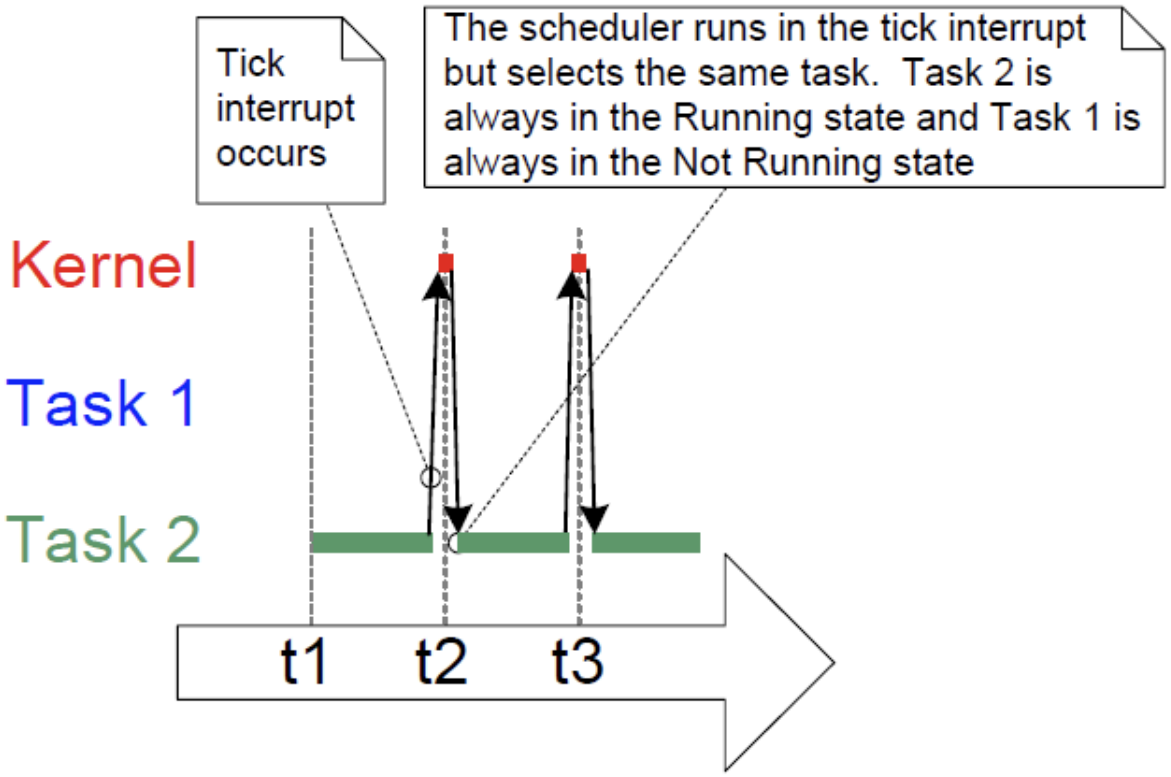
Task Management

- Task Delay
  - vTaskDelayUntil()
- Delay Until:
  - vTaskDelayUntil()

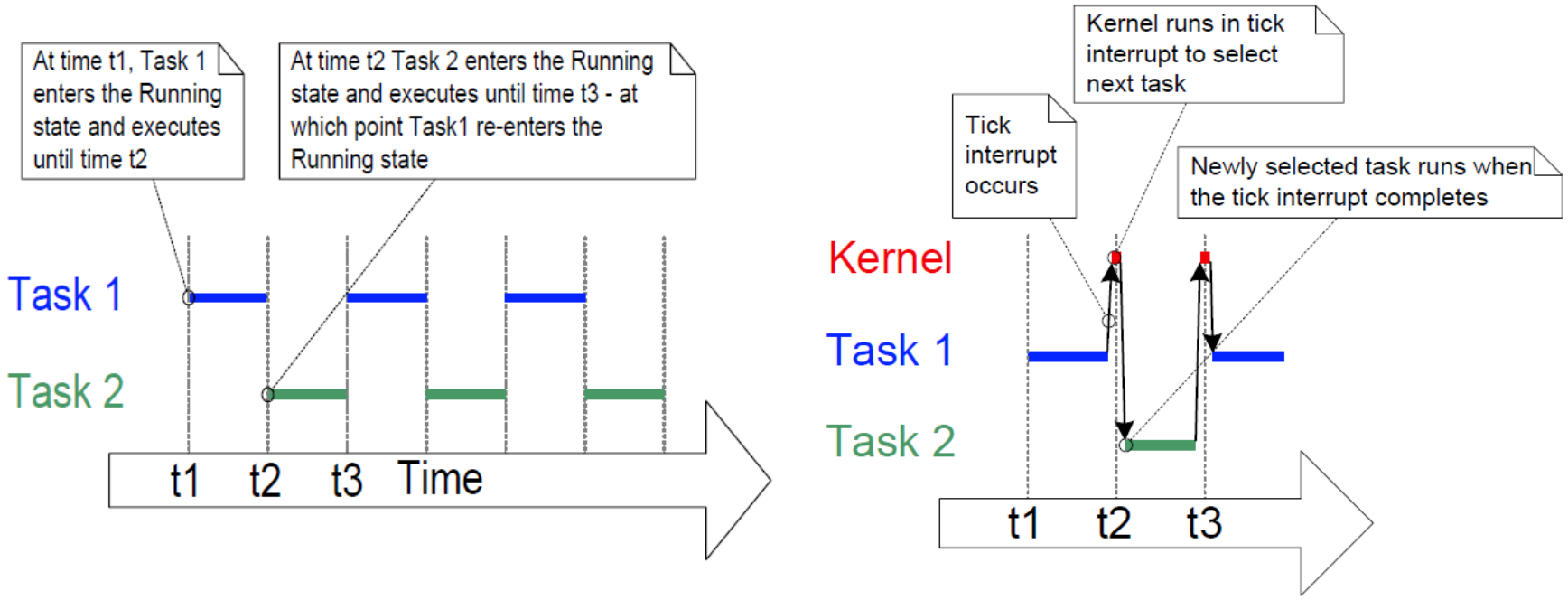
```
void vTaskFunction( void *pvParameters )
{
char *pcTaskName;
const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );

    /* The string to print out is passed in via the parameter.  Cast this to a
    character pointer. */
    pcTaskName = ( char * ) pvParameters;

    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; )
    {
        /* Print out the name of this task. */
        vPrintString( pcTaskName );

        /* Delay for a period.  This time a call to vTaskDelay() is used which places
        the task into the Blocked state until the delay period has expired.  The
        parameter takes a time specified in 'ticks', and the pdMS_TO_TICKS() macro
        is used (where the xDelay250ms constant is declared) to convert 250
        milliseconds into an equivalent time in ticks. */
        vTaskDelay( xDelay250ms );
    }
}
```

Task Scheduling

- How to schedule multiple tasks?
  - Time slicing Scheduling: per tick
  - Pre-emptive Scheduling: Priority-based

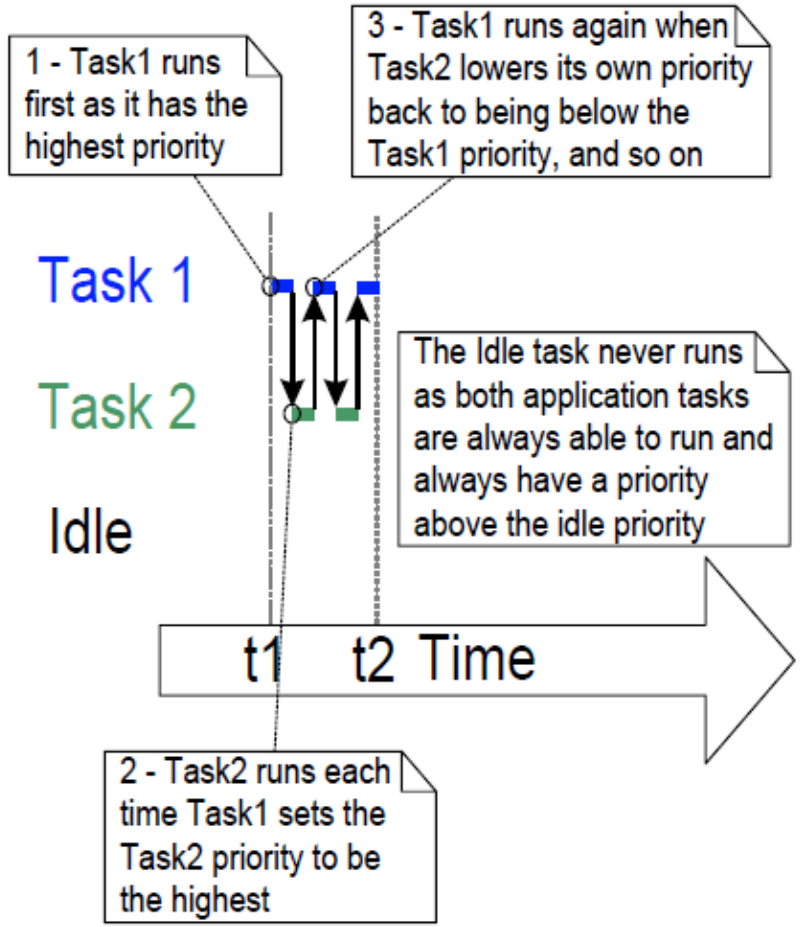


Time-slicing Task Scheduling



Priority Task Scheduling

```
/* Declare a variable that is used to hold the handle of Task 2. */
TaskHandle_t xTask2Handle = NULL;

int main( void )
{
    /* Create the first task at priority 2.  The task parameter is not used
    and set to NULL.  The task handle is also not used so is also set to NULL. */
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 2, NULL );
    /* The task is created at priority 2 ______^. */

    /* Create the second task at priority 1 - which is lower than the priority
    given to Task 1.  Again the task parameter is not used so is set to NULL -
    BUT this time the task handle is required so the address of xTask2Handle
    is passed in the last parameter. */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, &xTask2Handle );
    /* The task handle is the last parameter _____^^^^^^^^^^^^^ */

    /* Start the scheduler so the tasks start executing. */
    vTaskStartScheduler();

    /* If all is well then main() will never reach here as the scheduler will
    now be running the tasks.  If main() does reach here then it is likely there
    was insufficient heap memory available for the idle task to be created.
    Chapter 2 provides more information on heap memory management. */
    for( ;; );
}
```



Example Scheduling Question

- The following FreeRTOS program constraints create and executes 3 tasks T1, T2 and T3 of equal priority 1. FreeRTOS tick time is 5msec. Analyze the constraints and fill in the task timing diagram below until 90msec has expired.
- A. T1 is created first, T2 is created second and T3 is created third.
- B. At the end of the first execution of T2, T2 vTaskDelayUntil with argument of 30msec.
- C. At the end of the second execution of T1, T1 uses vTaskDelay for 15msec and raises T3 priority to 2.
- D. At the end of the second execution of T3, T3 vTaskDelay for 10msec.
- E. At the end of the fourth execution of T3, T2 vTaskDelay for 15msec, and T3 is suspended for 10msec.
- Place an (X) in the block if the task is running.
- Place an (S) in the box if the task is suspended.
- Place a (D) in the box if the task is delayed.